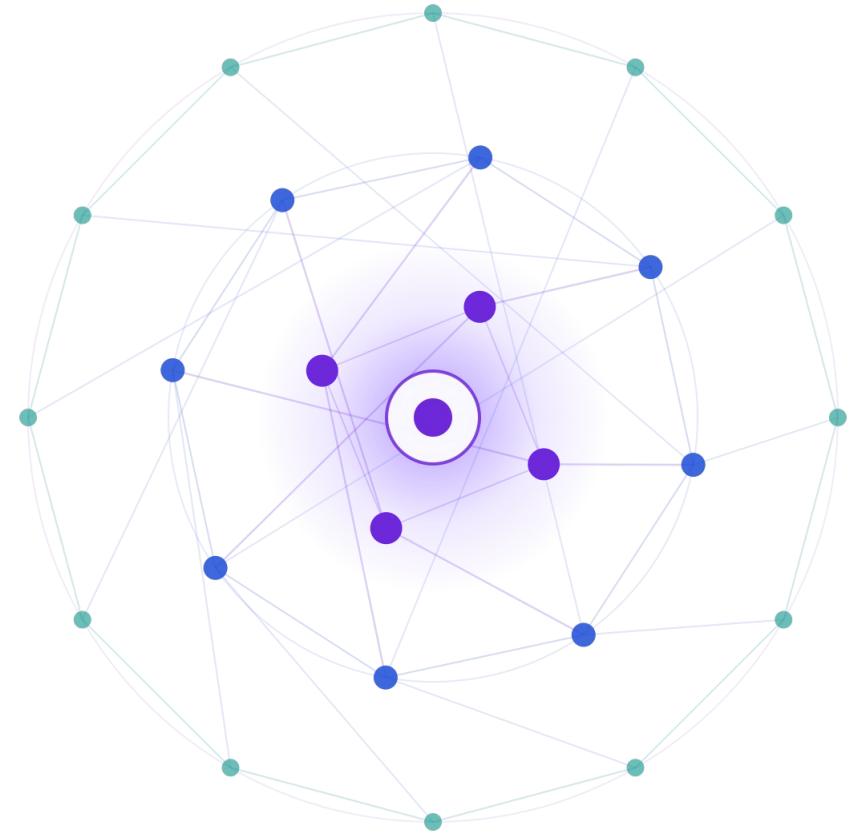


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - **Theme 02** • Smart Guided Troubleshooting Engine
- Team Name - **DeadlockDodgers**
- College Name - **Thapar**
- Member Name & Email 1- **Vyom Khanna** • vkhanna_be25@thapar.edu
- Member Name & Email 2- **Sanyam Ahuja** • sahuja2_be25@thapar.edu
- Member Name & Email 3- **Shauryajit Singh** • ssingh4_be25@thapar.edu
- Member Name & Email 4- **Parin Gupta** • pgupta_be25@thapar.edu
- Submission Github link - github.com/Sanyam-Ahuja/prism



Theme

Theme 02 · Smart Guided Troubleshooting Engine: the problem in our words, and how we answer each ask

The problem, in our words

Galaxy owners describe what they feel, not what is wrong. A support agent turns that into troubleshooting steps by hand (about 15 minutes per scenario, across millions of calls), and the user still has to hunt through Settings to act on the advice.

We want the complaint in, and a checked plan out: ordered steps taken from the support article, each settings step one tap from the right screen.

How people actually type it (all answered in our demo)

“tapping takes forever to register”

“phone lags when i tap anything”

“dropped it and screen cracked. also touch is slow af”

“battery dies so fast lol”

The theme asks for	What we built
Query enrichment: normalise a raw complaint	Normalise, split into separate problems, match by meaning against 130 LLM-written phrasings; 8–10 query_variations in every response
A two-stage LLM engine returning ordered steps as JSON	Stage 1: a strong LLM compiles checked plans offline. Stage 2: a local 1.5B LLM plans new articles live. Least disruptive step first
Map each step to the exact Settings deeplink	Hybrid BM25 + dense retriever over the 578-entry catalog; deeplinks are copied, never written
Fast-path cache under 300 ms	13.7 ms P95 on wording it has never seen
REST API; mapping reusable for 10k+ scenarios	POST /v1/troubleshoot. A new scenario = compile one article; no retraining
Report step accuracy, latency, cost	2.92–2.97 of 3 · 4.4 s P95 live · \$0.00 per query

Existing Solutions & Gaps

Every current option gives up one of: speed, grounding in the article, or a one-tap fix

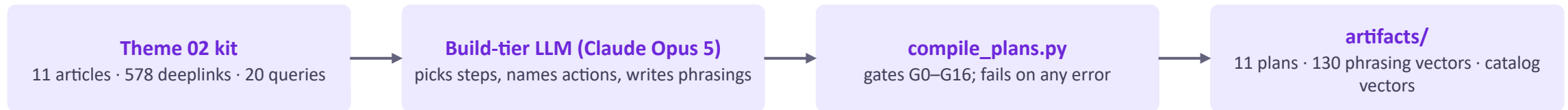
Existing approach	How it works	The gap
Human support agent	Reads the complaint, finds the article, writes the steps by hand	About 15 minutes per scenario; the user still hunts through Settings
Help-centre search / FAQ	Keyword search over support articles	Vague wording misses; the user reads a whole article; no one-tap links
Generic LLM chatbot	The model writes the steps and links itself	Our test, same model: only 47% of written steps were the article's own words; 2.3% of written deeplinks were correct
Keyword rules for Settings links	Match screen names to the catalog (BM25)	Our test: 13.3% right on free-form screen descriptions (hybrid retrieval: 66.7%)

The gap we close: answers in milliseconds from a cache, or seconds from a local model; every step is the article's own text; every deeplink comes from the catalog; every response is checked before it leaves.

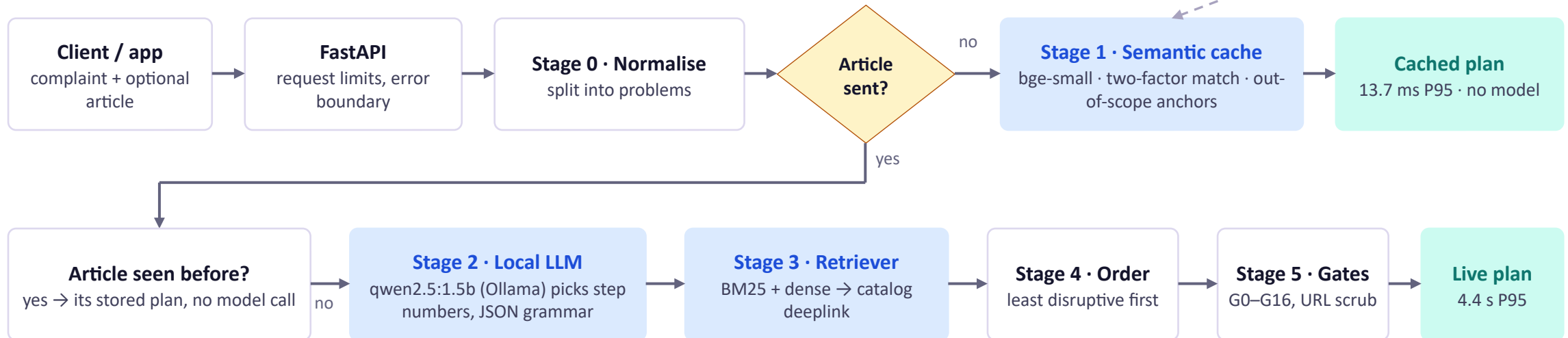
Our Solutions & Architecture Diagram

One pipeline, run twice: once offline to build a cache of checked plans, and live for articles it has not seen

BUILD TIME · OFFLINE, ONCE PER ARTICLE



RUNTIME · LOCAL, PER REQUEST (POST /v1/troubleshoot)



Output: schema-valid JSON with every action, its steps and deeplinks, plus query_variations and meta (latency, cache hit, model, cost, tokens). Out of scope or no valid plan → an honest empty-plan fallback. No hosted model is called at runtime.

Demo & Product Walkthrough

The real system on a laptop: python scripts/demo.py opens a demo page on the actual API

The screenshot shows the 'Smart Guided Troubleshooting Engine' interface. At the top, it says 'A vague complaint in, a validated one-tap troubleshooting plan out'. The interface is divided into several sections:

- DEMO SCENES:** A list of 8 scenarios, including 'Known issue', 'Same issue, new words', 'Another issue', 'Two problems', 'Never-seen issue', 'Out of scope', 'Planted URL', and 'Spare: camera'.
- CUSTOMER COMPLAINT:** A text input field containing 'tapping takes forever to register'.
- KNOWLEDGE-BASE ARTICLE (OPTIONAL):** A dropdown menu set to 'None — rely on the cache'.
- Build troubleshooting plan:** A button to generate the plan.
- Performance Metrics:** A row of four boxes showing 'PATH: Cache hit', 'LATENCY: 30 ms', 'TOKENS: 0', and 'COST: \$0.00'.
- TOUCH RESPONSE DELAY:** A section titled 'Follow these steps to perform this Touchscreen Response Troubleshooting' with a 'Confidence 82%' bar. It contains four numbered steps: 'Enable Touch Sensitivity', 'Configure Navigation Bar Buttons', 'Remove Damaged Screen Protector', and 'Try a Different Charger'.

Walkthrough: typed like a real user

- 1 “tapping takes forever to register”
cached plan in milliseconds
- 2 “phone lags when i tap anything”
same plan, different words
- 3 “screen flickering like crazy”
a different plan
- 4 “dropped it and screen cracked. also touch is slow af”
two problems, two plans
- 5 “battery dies so fast lol” + battery article
live plan from the local model
- 6 Same article again
stored plan, instant
- 7 “earbuds wont connect”
out of scope: honest fallback
- 8 URL planted in the article
1 URL in → 0 in the response

Demo video (2 min 52 s): Thapar_DeadlockDodgers_Demo.mp4 (repo root)

Tools and tech stack used

Small, local and open: everything runs on one laptop (i5-13420H, RTX 4050 6 GB)

Models

- qwen2.5:1.5b (4-bit) via Ollama on the GPU: picks steps for new articles
- BAAI/bge-small-en-v1.5 on the CPU: embeddings for the cache and retriever
- Claude Opus 5: offline build tier only, never at runtime

API & runtime

- Python 3.12 · FastAPI · Uvicorn
- Pydantic v2 with the supplied schema.py, unmodified
- httpx client for the local Ollama server

Retrieval & cache

- Hybrid rank-bm25 + dense cosine over 578 deeplinks
- Semantic cache over 130 phrasings (NumPy)
- Out-of-scope anchors and multi-intent splitting

Guardrails

- JSON-schema-constrained decoding (Ollama format)
- 17 gates G0–G16 and a URL scrubber on every plan
- Deterministic decoding: temperature 0, fixed seed

Testing & measurement

- pytest: 81 regression tests
- Scripts for accuracy, latency, ablations and HTTP stress
- python scripts/run_reports.py regenerates every number

Packaging & demo

- Dockerfile + docker-compose (API + Ollama)
- Offline image: encoder weights baked in
- Demo page (HTML/JS); video recorded with Playwright

Impact & Use case

Who uses it, and what changes for them

1

Contact-centre agent assist

The agent pastes the customer's words and gets an ordered, checked plan in milliseconds, instead of writing one by hand (about 15 minutes per scenario today).

13.7 ms

P95 cached answer

\$0.00

per query: all local

2

Self-service in the phone

In a support app or the voice assistant, the user taps a step and the exact Settings screen opens. Each step's validationDeeplink lets the app check the setting's current value.

70 req/s

on a laptop, 8 clients

0

invented steps or links

3

Support-content onboarding

A new support article becomes a plan in 2.6 s (median) with no retraining, and the plan is kept for every later request for that article.

Built for 10k+ scenarios

Plans are keyed by article and deeplinks come from the catalog, so adding a scenario means compiling one article. No model is fine-tuned.

Innovation highlights, results and limitations

Measured on the laptop with nothing else running; raw output in reports/, full report in docs/metrics.md

Innovation highlights

The model picks, never writes. It returns step numbers; code copies the article's steps verbatim.

No model touches a deeplink. A hybrid retriever maps each screen to a catalog URI.

The cache is the pipeline, frozen. Cached plans are its own checked output.

Two problems, two plans; off-topic complaints get an honest fallback (37 of 38).

Casual wording works. Matching is by meaning, so “touch is so slow” finds the same plan as the formal query.

Result	Target	Ours
Cache hit, P95	≤ 300 ms	13.7 ms
Live plan, P95	≤ 8 s	4.4 s (P50 2.6)
Schema-valid	≥ 99%	100% (325/325)
Web-URL leaks	0	0
Deeplinks from catalog	100%	100%
Hit rate, new wording	≥ 80%	84.6% · 100%*
Step accuracy (0–3)	rubric	2.92 · 2.97 live
Deeplink relevance (0–2)	rubric	1.78
Cost per query	tracked	\$0.00

* 84.6% on the tuning set, 100% on 27 phrasings written afterwards

Limitations

- Cached plans cover Display only: all supplied data is Display.
- Live plans link a specific screen for 19% of settings actions (69% for cached plans).
- In a two-problem complaint, a problem no plan covers is dropped without notice.
- A new article always takes the live path, even when a cached plan fits.
- Very vague wording (“my phone is being weird”) gets the fallback; two problems need a clear break between them.

What's next

From a laptop prototype to a worklet: each step builds on what is already measured

1

Cover Battery, Camera, Performance

Compile plans from their articles. The pipeline already maps their screens: 100% on 26 labelled deeplinks.

2

Device-state-aware plans

Read each step's validationDeeplink on the phone and skip steps whose setting is already right.

3

Say what went unanswered

Tell the caller when one problem in a complaint had no plan, instead of dropping it.

4

More one-tap links in live plans

Raise specific-screen coverage from 19% toward the cached plans' 69% with better screen descriptions.

5

On-device

Measure the 1.5B model on a Galaxy phone's CPU/NPU, so plans work offline.

6

More languages

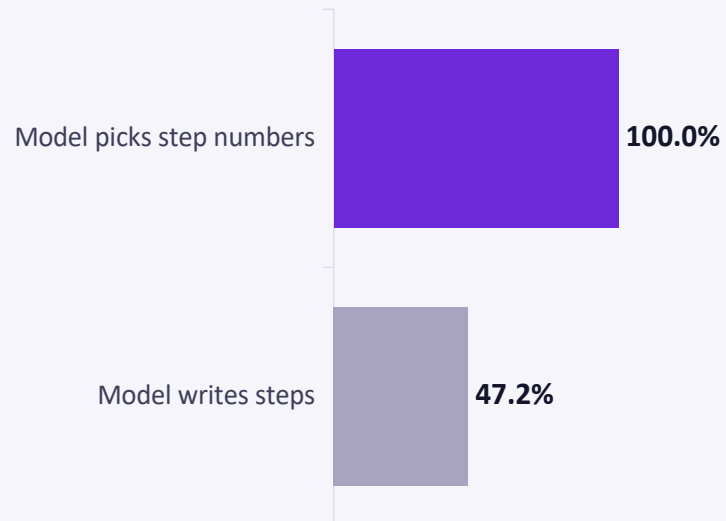
Hindi and code-mixed Hinglish complaints, via multilingual embeddings.

Brownie points slide(differentiation)

We measured the design choices, not just the system: same 1.5B model, same 11 articles

Steps: pick, don't write

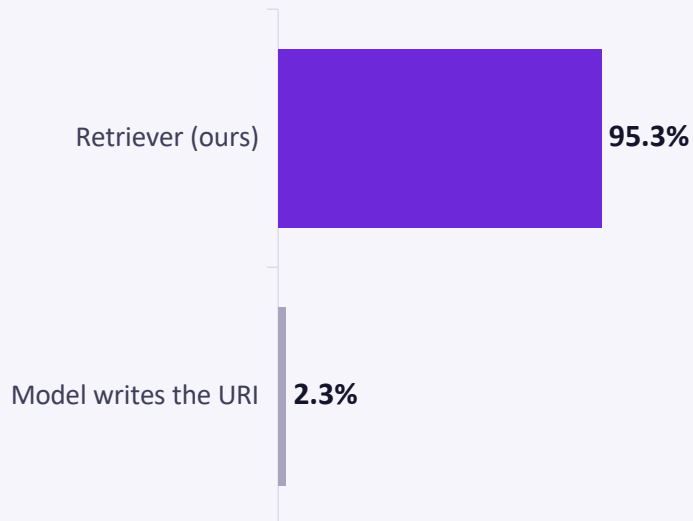
Plan steps that are the article's own words



- Writing: 61% more tokens and 10.1 s P95, over the 8 s budget (picking: 4.5 s).

Deeplinks: retrieve, don't write

Screens linked correctly (43 hand-labelled)



- Catalog URIs are masked: a model writing them invents 42 of 43.

Also different

- Seven local models benchmarked on the same inputs before choosing qwen2.5:1.5b.
- Test sets written before evaluation, separate from the tuning set.
- Every number regenerates with one command: `python scripts/run_reports.py`.
- Offline container; no hosted model or API key at runtime.
- AI use disclosed in AI_DISCLOSURE.md.

Checklist- Updated on Public GitHub

- Working prototype code — public or shared GitHub repo (Y/N) — Y: github.com/Sanyam-Ahuja/prism (tag **PRISM_GENAI_HACKATHON_Y2026**)
- README with reproducible setup instructions (Y/N) — Y: **README.md (pip or Docker Compose)**
- Demo video, max 5 minutes (YouTube or Drive link) — Y: **2 min 52 s · Thapar_DeadlockDodgers_Demo.mp4 (repo root)**
- Presentation file (PPT or PDF) (Y/N) — Y: **Thapar_DeadlockDodgers_Submission.pptx (+ PDF), repo root**

Also in the tagged commit: Dockerfile and docker-compose.yml · requirements.txt (also requirement.txt) and requirements.lock · AI_DISCLOSURE.md · reports/ with the raw output behind every number

Thank you

DeadlockDodgers · Thapar · github.com/Sanyam-Ahuja/prism

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India